Experiment No. 7

Experiment Title	To implement the Clustering algorithms (k-means, Agglomerative, DBSCAN) using Python
Course Outcomes	CO4: Implement the appropriate data mining methods like classification, clustering or Frequent Pattern mining on large data sets. CO5: Define and apply metrics to measure the performance of various data mining algorithms.
Date of Preparation	
Date of Submission	
Signature & Grade	

1. Aim

To implement the K-Means, Agglomerative (hierarchical) and DBSCAN clustering algorithms in Python (scikit-learn) on an airline passenger dataset, and to compare them using the silhouette score, the Davies-Bouldin index, the Calinski-Harabasz index and the cluster sizes.

2. Theory

2.1 Clustering

Clustering is an unsupervised data mining technique. It groups records so that records in the same cluster are similar to each other and different from records in other clusters, without using any class label. Because it works with distances, the attributes must be on the same scale.

2.2 The Three Algorithms

Algorithm	How it works	Needs	Limitation
K-Means	Picks k centres, assigns each record to the nearest centre, then moves the centres until they settle.	Number of clusters k	Finds round clusters of similar size; sensitive to outliers.
Agglomerative	Starts with every record as its own cluster and repeatedly merges the two closest clusters, giving a tree called a dendrogram.	A linkage method and a cut of the tree	Memory and time grow with the square of the number of records.
DBSCAN	Joins points that have enough close neighbours into dense clusters and labels isolated points as noise.	eps (radius) and min_samples	Hard to tune eps; finds one cluster when the density is uniform.

2.3 Evaluation Measures

Measure	Meaning
Inertia (elbow method)	Total squared distance of records from their cluster centre; plotted against k to look for a bend.
Silhouette score	How well each record fits its own cluster compared with the nearest other cluster. It lies between −1 and 1; higher is better.
Davies-Bouldin index	Average similarity between each cluster and its most similar cluster; lower is better.
Calinski-Harabasz index	Ratio of between-cluster spread to within-cluster spread; higher is better. It grows with the number of records.
k-distance plot	Sorted distance of every point to its k-th nearest neighbour; its elbow suggests eps for DBSCAN.

3. Dataset and Tools

Dataset: Airline Passenger Satisfaction (Kaggle), file train.csv, with 103,904 passengers. Six numeric features were used: Age, Flight Distance, Inflight wifi service, Online boarding, Seat comfort and Inflight entertainment (the four ratings are from 0 to 5). The satisfaction label was not used.

Source: kaggle.com/datasets/teejmahal20/airline-passenger-satisfaction

K-Means was run on all 103,904 records. Agglomerative clustering was run on a random sample of 3,000 records and DBSCAN on a random sample of 10,000, because Agglomerative clustering needs memory that grows with the square of the number of records.

Tools: Python 3 with pandas, NumPy, Matplotlib, Seaborn, SciPy and scikit-learn.

4. Implementation

4.1 Select, Scale and Reduce the Features

```
features = ['Age', 'Flight Distance', 'Inflight wifi service',
            'Online boarding', 'Seat comfort', 'Inflight entertainment']
X_imp = SimpleImputer(strategy='median').fit_transform(df[features])
scaler = StandardScaler()
X = scaler.fit_transform(X_imp)
X_pca = PCA(n_components=2).fit_transform(X)      # only for drawing charts
```

```
Features used: ['Age', 'Flight Distance', 'Inflight wifi service', 'Online boarding', 'Seat comfort', 'Inflight entertainment']
Records: 103904 | Features: 6
Missing values in these features: 0
After scaling: mean = [ 0.  0.  0. -0. -0. -0.] | std = [1. 1. 1. 1. 1. 1.]
PCA (for plotting only): PC1 = 36.6%, PC2 = 17.8% of the variance
```

	mean	std	min	max
Age	39.38	15.11	7.0	85.0
Flight Distance	1189.45	997.15	31.0	4983.0
Inflight wifi service	2.73	1.33	0.0	5.0
Online boarding	3.25	1.35	0.0	5.0
Seat comfort	3.44	1.32	0.0	5.0
Inflight entertainment	3.36	1.33	0.0	5.0

Figure 1: Features used, scaling check, PCA variance and summary statistics

The six features have no missing values. After standardisation each has a mean of 0 and a standard deviation of 1, so no feature dominates the distance calculation. PCA is used only to draw the clusters on a 2-D chart. The two components hold 36.6% and 17.8% of the variance (54.4% together), so the 2-D charts show about half of the information in the data.

4.2 K-Means: Choose the Number of Clusters

```
inertia = [KMeans(n_clusters=k, n_init=10).fit(X).inertia_ for k in range(1, 11)]
labels = KMeans(n_clusters=k, n_init=10).fit_predict(X)
silhouette_score(X, labels, sample_size=10000)
```

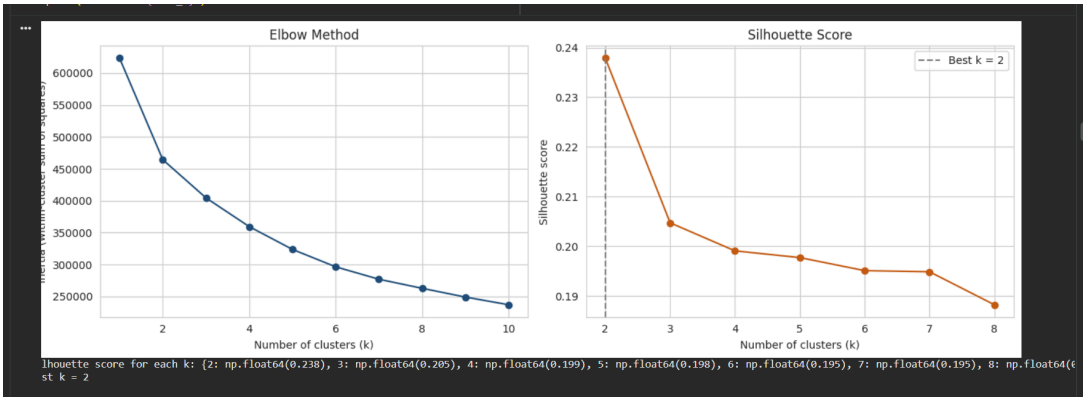


Figure 2: Elbow method and silhouette score for different numbers of clusters

The elbow curve falls steadily and has no sharp bend, so it does not point to a clear k. The silhouette score is highest at k = 2 (0.238) and then falls (0.205 for k = 3, 0.199 for k = 4, and about 0.19 to 0.20 for k = 5 to 8). Two clusters were therefore chosen.

4.3 K-Means Clustering

```
kmeans = KMeans(n_clusters=2, n_init=10, random_state=42).fit(X)
centroids = scaler.inverse_transform(kmeans.cluster_centers_)
```

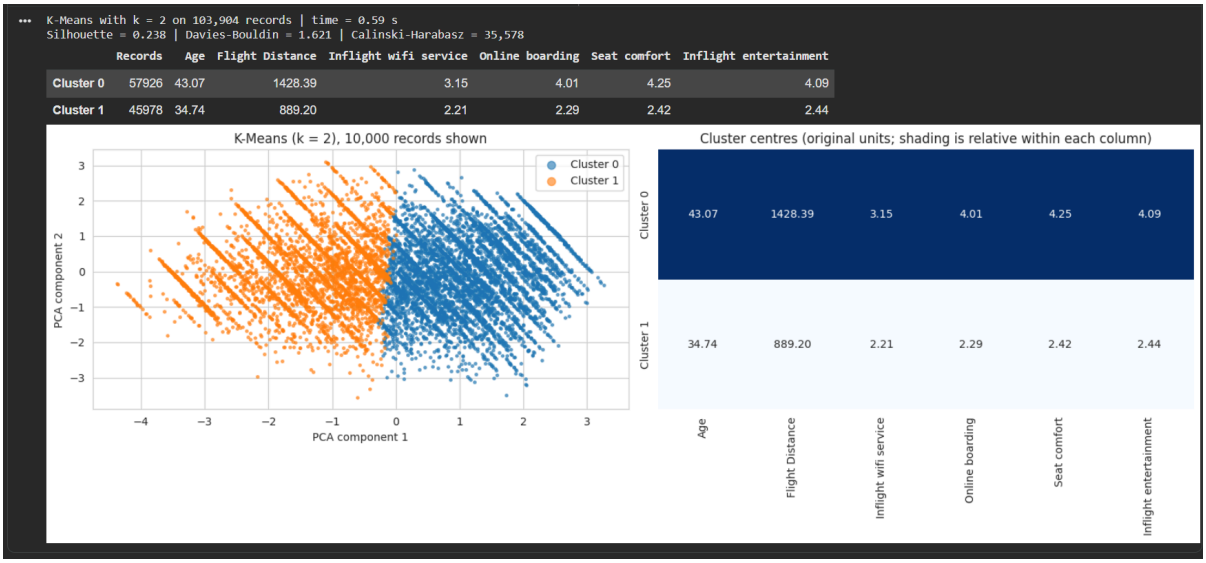


Figure 3: K-Means result — metrics, cluster centres, scatter plot and heatmap

K-Means with k = 2 on all 103,904 records took 0.59 seconds, with a silhouette score of 0.238, a Davies-Bouldin index of 1.621 and a Calinski-Harabasz index of 35,578. The two clusters are described by their centres:

Cluster	Records	Profile (from the cluster centres)
Cluster 0	57,926 (55.8%)	Older passengers (average age 43.1) on longer flights (1,428) who give high ratings: online boarding 4.01, seat comfort 4.25, entertainment 4.09, wifi 3.15.
Cluster 1	45,978 (44.2%)	Younger passengers (average age 34.7) on shorter flights (889) who give low ratings of about 2.2 to 2.4 for every service.

The scatter plot shows the two clusters separated mainly along the first PCA component, with an overlap in the middle. The diagonal stripes appear because the ratings are whole numbers. The silhouette score is low (the maximum is 1), which agrees with the overlap: the two groups differ gradually and are not sharply separated.

4.4 Agglomerative Clustering: Dendrogram

```
Z = linkage(X[idx], method='ward')
dendrogram(Z, truncate_mode='lastp', p=30)
```

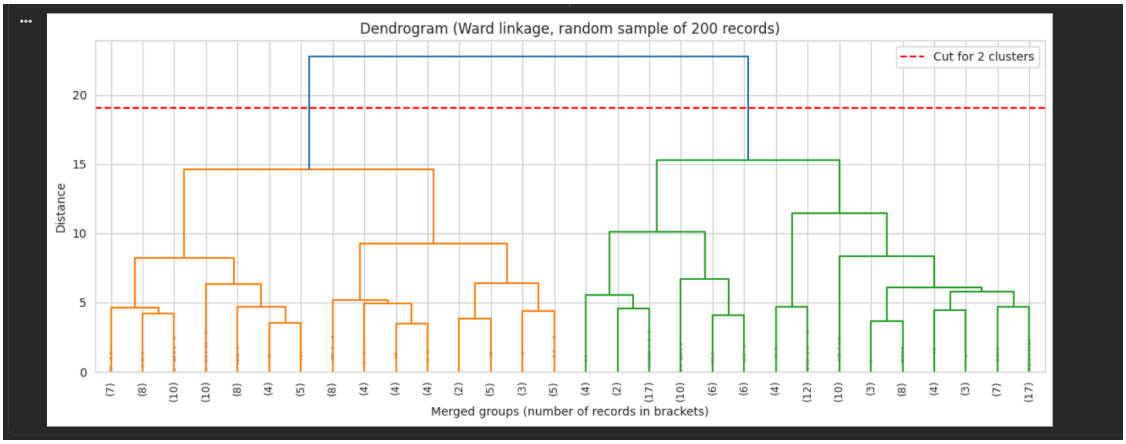


Figure 4: Dendrogram (Ward linkage, random sample of 200 records)

The two large groups join at the top at a distance of about 23, much higher than any other merge (the next highest are at about 15). Cutting the tree at the red line therefore gives two clusters, which agrees with the choice of k = 2 for K-Means.

4.5 Agglomerative Clustering Result

```
AgglomerativeClustering(n_clusters=2, linkage='ward').fit_predict(X_agg)
```

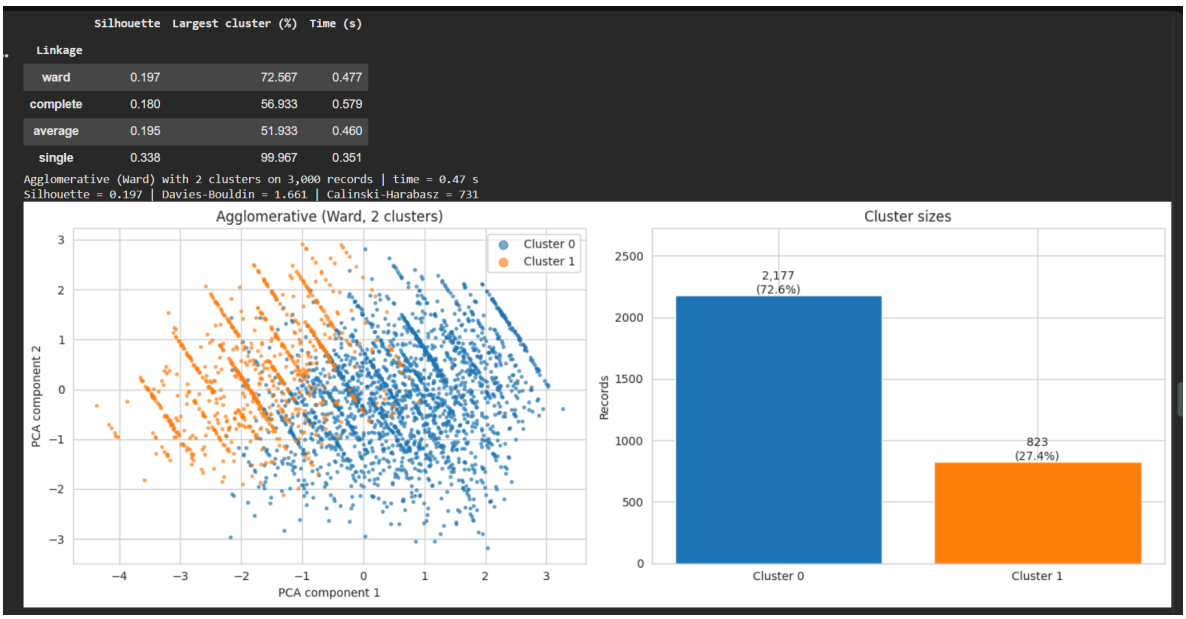


Figure 5: Comparison of linkage methods, Ward result, scatter plot and cluster sizes

Four linkage methods were compared on the same 3,000 records. Single linkage has the highest silhouette (0.338), but it puts 99.97% of the records in one cluster (only one record is in the other), so that score is misleading. Ward linkage was used. It gave clusters of 2,177 records (72.6%) and 823 records (27.4%), with a silhouette score of 0.197, a Davies-Bouldin index of 1.661 and a Calinski-Harabasz index of 731, in 0.47 seconds. Its smaller cluster (27.4%) is smaller than the second K-Means cluster (44.2%).

4.6 DBSCAN: Choose eps

```
dist, _ = NearestNeighbors(n_neighbors=12).fit(X_db).kneighbors(X_db)
kdist = np.sort(dist[:, -1])
```

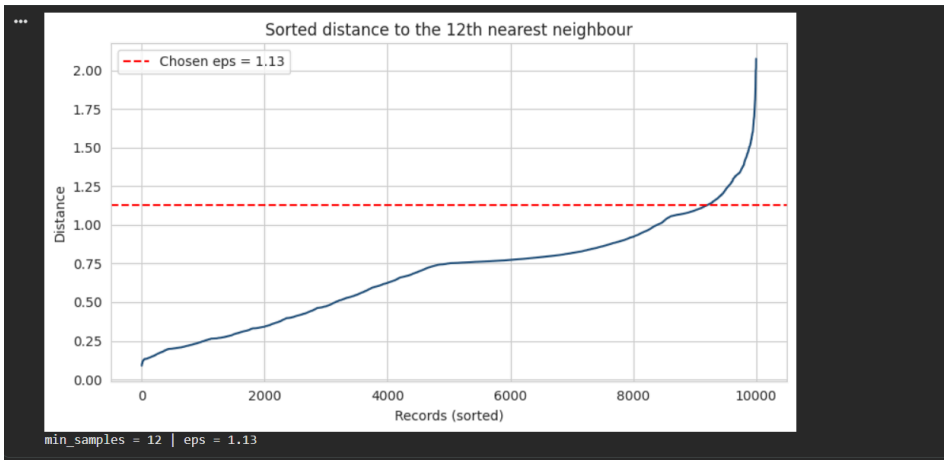


Figure 6: Sorted distance to the 12th nearest neighbour

min_samples was set to 12 (2 × the 6 features). The distance rises slowly from about 0.1 to about 1.0 and then sharply for the last few hundred records, which are isolated points. The elbow of the curve gives eps = 1.13.

4.7 DBSCAN Clustering Result

```
db_labels = DBSCAN(eps=1.13, min_samples=12).fit_predict(X_db)
```

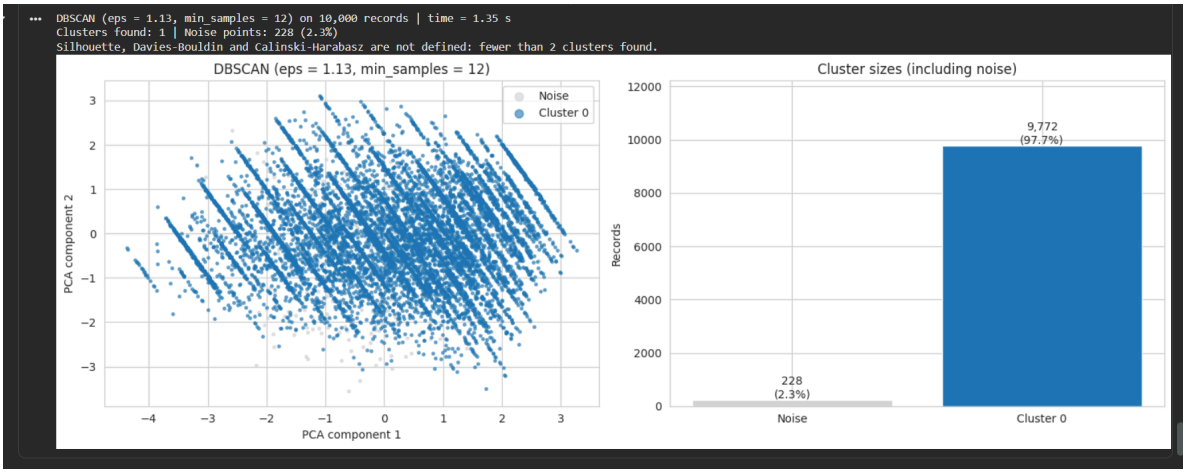


Figure 7: DBSCAN result — clusters, noise points and cluster sizes

On 10,000 records DBSCAN found only one cluster (9,772 records, 97.7%) and labelled 228 records (2.3%) as noise. The silhouette, Davies-Bouldin and Calinski-Harabasz scores are not defined for a single cluster. The ratings are whole numbers, so the points lie on stripes in the chart, and these stripes join into one continuous dense region. DBSCAN therefore does not split the passengers into separate groups here; it only picks out some unusual passengers at the edges as noise.

4.8 Compare the Algorithms

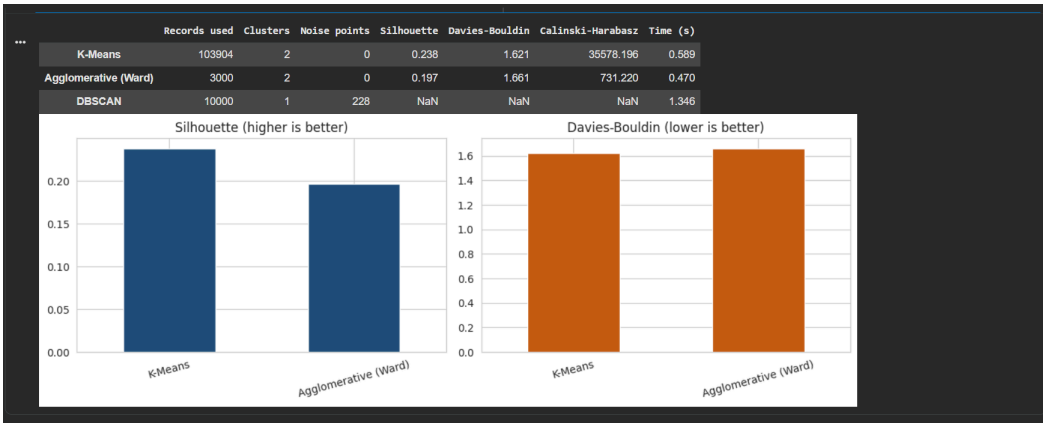


Figure 8: Comparison table and bar charts of the three algorithms

K-Means has the best silhouette score (0.238 against 0.197 for Agglomerative) and a slightly lower Davies-Bouldin index (1.621 against 1.661). These scores must be compared with care, because the algorithms were run on different numbers of records, and the Calinski-Harabasz index grows with the number of records, so the values 35,578 and 731 cannot be compared with each other.

5. Result Comparison

	K-Means	Agglomerative (Ward)	DBSCAN
Records used	103,904	3,000 (sample)	10,000 (sample)
Parameters	k = 2	2 clusters, Ward linkage	eps = 1.13, min_samples = 12
Clusters found	2	2	1
Cluster sizes	57,926 / 45,978	2,177 / 823	9,772
Noise points	0	0	228 (2.3%)
Silhouette	0.238	0.197	not defined
Davies-Bouldin	1.621	1.661	not defined
Calinski-Harabasz	35,578	731	not defined
Time (s)	0.59	0.47	1.35

6. Conclusion

K-Means, Agglomerative clustering and DBSCAN were implemented in Python on six standardised features of the Airline Passenger Satisfaction data. K-Means (k = 2, chosen by the silhouette score) gave the best result, with a silhouette score of 0.238: it split the passengers into older passengers on longer flights who rate the services highly (55.8%) and younger passengers on shorter flights who rate them low (44.2%). Agglomerative clustering also gave two clusters but could only be run on a sample, and DBSCAN found just one dense cluster with 2.3% noise. The low silhouette scores show that the passenger groups overlap and are not sharply separated.